

Unseen Assessment Key

Unseen Assessment — Instructor Answer Key & Grading Guide

Model answers for `dnp3_assessment.pcap` and `mqtt_assessment.pcap`. Frame numbers are exact (verified with `tshark`). Award points on the four-criterion rubric (below); enforce the two identification gates.

Part A — `dnp3_assessment.pcap` (30 frames, 3 TCP streams)

A1 — actors.

Source IP	DNP3 link addr	Role	Evidence
10.30.0.5	100	Primary master	Sends READ integrity/Class-0 polls (frames 4, 8) and a CONFIRM (frame 14)
10.30.0.6	101	Secondary read-only master	Sends only a READ Class-0 poll (frame 19); issues no controls or confirms
10.30.0.66	10	Neither — impersonates the outstation	Sends an UNSOLICITED RESPONSE (frame 26) claiming to be the outstation (link 10)

The real outstation is 10.30.0.20 (link 10), which answers the polls (frames 6, 10, 21) and sends the one **genuine** unsolicited event (frame 12).

A2 — the malicious frame: frame 26, an UNSOLICITED RESPONSE (function 0x82). Tells (any three): 1. **IP source 10.30.0.66 ≠ the real outstation's 10.30.0.20**, yet its DNP3 link source claims **10** (the outstation) — IP/link-owner mismatch. 2. It rides a **separate TCP stream** (to the master's port) from the real outstation's session. 3. Its reported **breaker = open / 58.50 Hz contradicts** the polls seconds earlier (frames 6/10, breaker closed / 60.00 Hz) **and** the parallel secondary-master read (frame 21). 4. It is a **response (0x82), not a control** — a “watch the SELECT/OPERATE” detection never fires on it. (*Contrast: frame 12 is the genuine unsolicited — same link 10, but IP source 10.30.0.20, and consistent data.*)

A3 — the lie vs. the truth. Frame 26 paints the breaker as **OPEN (point 0 = 0, “tripped”)** and frequency as **58.50 Hz** (false under-frequency, value 5850). The **true** state from the legitimate polls: breaker **CLOSED** (binary input point 0 = 1 in frames 6/10/21) and frequency **60.00 Hz** (analog = 6000). It is a loss-of-view / spoofed-reporting attack (MITRE ATT&CK for ICS **T0856**), inviting an unnecessary operator response.

A4 — why the proposed rule fails, and the fix. Two independent reasons: 1. The attack is an **UNSOLICITED RESPONSE, not a control** — the rule’s function-code filter (SELECT/OPERATE/...) never matches, so it is a guaranteed false-negative here. 2. It hard-codes a single master IP and **ignores the legitimate secondary master 10.30.0.6**, so it is simultaneously too narrow (misses spoofed responses) and, if broadened to “not a master,” would false-positive on 10.30.0.6. **Rewrite:** alarm when a given **DNP3 link address appears from an unexpected source IP / TCP session** (e.g., link 10 arriving from anything other than 10.30.0.20), and/or when an **unsolicited response’s IP source ≠ the known outstation IP** — with the legitimate master **set** {10.30.0.5, 10.30.0.6} allow-listed.

Part B — mqtt_assessment.pcap (66 frames)

B1 — the harvest. Frames **47, 49, 51** (broker → 10.40.0.66), delivered immediately after the rogue’s SUBACK. The field proving they are stored, not live: the PUBLISH **RETAIN flag = 1** (`mqtt.retain==1`) — live telemetry in this capture (e.g., frames 34, 55) has **RETAIN = 0**. Harvested topics and leaked values: - `plant/line1/config` → `hi_setpoint 80`, firmware `1.4.2`, mode `auto` - `plant/line1/command` → the command **schema and topic name** (`{"actuator":"pump1","cmd":"STOP"}`) - `plant/site/info` → gateway `edge-gw-3` and a **service username** `svc_ingest`

B2 — authN vs. authZ. Yes, the broker authenticated the rogue — the anonymous CONNECT is **frame 39** (client `mqtt-recon`, no username/password) and the broker **accepted** it with CONNACK **return code 0** in **frame 41**. So the harvest and the injection are **authorization** failures, not authentication: - The **harvest** is a missing **read ACL** — the setting that permitted the accept is `allow_anonymous true`; the setting that permitted the reads is the absence of a per-topic read ACL. - The **injection** (frame 57, PUBLISH to `plant/line1/command`) is a missing **write ACL**. - The flag that makes it persist is **RETAIN = 1**. Because it is retained, the broker **stores it, replacing the legitimate “STOP”**; when the pump controller next reconnects and subscribes `plant/line1/command`, the broker delivers the **malicious retained “START / valve open”** — so the pump starts on reconnect, long after the attacker has disconnected (frame 59).

B3 — why “reject #” fails, and the real fix. The `#`-block fails because: 1. The attacker **already harvested the exact topic names** (frames 47–51), so it can subscribe to each topic **directly** (`plant/line1/command`, ...) without ever using `#`. 2. The **injection is a PUBLISH, not a subscribe** — blocking wildcard subscriptions does nothing to a write. **The two controls that**

actually close it: (a) `allow_anonymous false + authentication` (client credentials or certs) so the rogue is refused at CONNECT before any topic access; and (b) **per-topic read/write ACLs bound to identity**, so even an authenticated client cannot read `config / command / site` or write to a command topic. (Defense in depth: disable retained messages on command topics, or require a retained-message purge policy.)

Rubric (0–3 each, 12 total) and mastery gates

Score each of the four criteria — **Identify · Interpret · Justify · Recommend-control** — using the student worksheet's rubric table. **Mastery = $\geq 10/12$, no criterion < 2 , AND both gates:** (A) student names **frame 26, the spoofed UNSOLICITED RESPONSE**, as the one malicious DNP3 frame; (B) student identifies the **retained command PUBLISH (frame 57)** as a persistent **write-authorization** abuse. Missing either gate → cap at *Approaching*. Bands: Mastery 10–12 (+gates) · Approaching 7–9 · Not yet ≤ 6 .

Common partial-credit notes. Calling frame 12 (the genuine unsolicited) the attack is the classic Identify=0 error — both share link 10, but only frame 26 has the IP mismatch. Saying “MQTT has no authentication” is the classic Justify miss — the rogue *was* authenticated (frames 39/41); the failure is authorization. Proposing TLS as the harvest fix earns partial credit only: TLS stops an on-path sniffer but not an **authenticated** rogue reading via the broker — the gap here is ACLs, not encryption.